

Packages à prioriser

Atelier intensif

Guillaume Boucher

INSPQ

Le 15 septembre 2026

Tidyverse

Les *packages* qui seront présentés font partie du **tidyverse**, un **ensemble de packages** R conçu pour la science des données.

Le *tidyverse* s'installe et se charge de la même manière qu'un *package* normal.

```
install.packages("tidyverse") # installation  
library(tidyverse)           # chargement
```

Puisque `library(tidyverse)` charge un ensemble de *packages*, on suggère d'appeler uniquement le ou les *packages* utilisés, surtout si vous avez des enjeux d'espaces mémoire.

Exécuter `library(tidyverse)` est équivalent à

```
library(ggplot2)  
library(dplyr)  
library(tidyr)  
library(readr)  
library(purrr)  
library(tibble)  
library(stringr)  
library(forcats)  
library(lubridate)
```

Concepts importants à connaître d'abord

Dans le tidyverse, il y a deux nouveaux concepts à connaître.

1) Les *pipes* (`%>%`, `|>`) insèrent automatiquement le résultat précédent comme premier argument de la fonction suivante.

```
mean(1:5)
```

```
#> [1] 3
```

```
1:5 |> mean()
```

```
#> [1] 3
```

```
1:5 %>% mean()
```

```
#> [1] 3
```

Concepts importants à connaître d'abord

2) L'évaluation des données (*data masking*)

Permet d'utiliser des variables directement à l'intérieur d'une table. Il n'est plus nécessaire d'indiquer le nom de la table (`ma_table$ma_variable`).

Pour calculer l'IMC :

```
# Sans data masking (R de base)
ma_table$imc <- ma_table$poids / ma_table$taille^2

# Avec data masking (tidyverse)
mutate(ma_table, imc = poids / taille^2)

# Avec data masking + pipes
ma_table %>% mutate(imc = poids / taille^2)
```

Table pour exemples

cohort_rx_dx

```
#>      id  sexe  rls      naiss      start  code duration  dx1  dx2
#>    <int> <char> <char>    <Date>    <Date> <char>    <num> <char> <char>
#>  1:      1      M  1111 1967-09-06 2000-01-03      C      1  <NA>  <NA>
#>  2:      1      M  1111 1967-09-06 2000-01-04      D      1  <NA>  <NA>
#>  3:      1      M  1111 1967-09-06 2000-01-07      S     30  <NA>  <NA>
#>  4:      1      M  1111 1967-09-06 2000-01-09      C      1  <NA>  <NA>
#>  5:      1      M  1111 1967-09-06 2000-01-11      C      1  <NA>  <NA>
#> ---
#> 17192:   100      M  1632 1990-11-03 2001-12-19      K     15  <NA>  <NA>
#> 17193:   100      M  1632 1990-11-03 2001-12-19      S     15  <NA>  <NA>
#> 17194:   100      M  1632 1990-11-03 2001-12-26      B     30  <NA>  <NA>
#> 17195:   100      M  1632 1990-11-03 2001-12-27      E     90  <NA>  <NA>
#> 17196:   100      M  1632 1990-11-03 2001-12-29      N      7  <NA>  <NA>
#>      dx3
#>    <char>
#>  1:  <NA>
#>  2:  <NA>
#>  3:  <NA>
#>  4:  <NA>
#>  5:  <NA>
#> ---
#> 17192:  <NA>
#> 17193:  <NA>
#> 17194:  <NA>
#> 17195:  <NA>
```

dplyr

Quand utiliser dplyr ?

- Aussitôt que vous êtes dans un `data.frame`
 - Pour manipuler et transformer **le contenu** d'un tableau.
 - Lorsque les données ont la bonne forme, mais qu'on veut extraire des informations ou faire des calculs.
- Filtrer, sélectionner, créer des colonnes, grouper, calculer des moyennes, trier...

Les principales fonctions sont :

- `select()` pour la sélection des variables (colonnes);
- `filter()` pour la sélection des observations (lignes);
- `mutate()` pour l'ajout ou la modification de variables;
- `group_by()` pour effectuer une opération *par groupe*;
- `summarise()` pour des statistiques résumées;
- `arrange()` pour trier les données.

La page officielle est disponible [ici](#).

dplyr contient combien d'objets (fonctions, variables, tables...) ?

```
length(ls("package:dplyr"))
```

```
#> [1] 288
```

Utiliser dplyr

```
library(dplyr)
```

```
#>  
#> Attachement du package : 'dplyr'  
  
#> Les objets suivants sont masqués depuis 'package:stats':  
#>  
#>      filter, lag  
  
#> Les objets suivants sont masqués depuis 'package:base':  
#>  
#>      intersect, setdiff, setequal, union
```

Ce message indique quelles sont les fonctions du *package* dplyr qui ont le même nom qu'une fonction d'un autre *package*.

L'utilisation des fonctions `filter` et `lag` sont maintenant par défaut celles de dplyr plutôt que stats.

Même chose pour les fonctions `intersect`, `setdiff`, `setequal` et `union` qui proviennent du *package* base (R de base).

Objets masqués

L'ordre dans lequel les *packages* sont appelés a un impact sur quel objet est masqué par quoi. Il est préférable de charger les *packages* au début de son code pour éviter l'erreur ci-dessous :

```
x <- 1:10  
filter(x, rep(1, 3)) # par défaut stats::filter()  
library(dplyr)  
filter(x, rep(1, 6)) # maintenant par défaut dplyr::filter()
```

Erreur dans UseMethod("filter") :
pas de méthode pour 'filter' applicable pour un objet de classe
"c('integer', 'numeric')"

Le message d'erreur indique qu'on a essayé d'utiliser `filter()` sur un objet numérique alors que la fonction s'attend à recevoir un tableau.

Note technique : les méthodes

Exemple avec la fonction `print()` :

```
print()  # fonction générique  
print.data.frame()  # Affichage des tableaux de données  
print.Date()  # Affichage des dates
```

Pour connaître toutes les méthodes d'une fonction :

```
methods(print)[1:10]
```

```
#> [1] "print.acf"           "print.activeConcordance"  
#> [3] "print.AES"          "print.all_vars"  
#> [5] "print.anova"        "print.any_vars"  
#> [7] "print.aov"          "print.aovlist"  
#> [9] "print.ar"           "print.Arima"
```

```
length(methods(print))
```

```
#> [1] 300
```

select

On utilise `select(.data, ...)` pour choisir ou exclure certaines colonnes.

Arguments

- **.data** : Un `data.frame` ou tout ce qui en découle. Voir `tibble`, `lazy data.frame`, `data.table`...
- **...** : Une ou plusieurs expressions séparées par des virgules.

Sélection simple

```
select(cohort_rx_dx, id, code, start, duration) # prescriptions des ids
```

```
#>      id    code      start duration
#>   <int> <char>   <Date>     <num>
#> 1:     1      C 2000-01-03         1
#> 2:     1      D 2000-01-04         1
#> 3:     1      S 2000-01-07        30
```

```
# Cette méthode sera priorisée dans les prochains exemples
cohort_rx_dx %>% select(id, code, start, duration)
```

```
#>      id    code      start duration
#>   <int> <char>   <Date>     <num>
#> 1:     1      C 2000-01-03         1
#> 2:     1      D 2000-01-04         1
#> 3:     1      S 2000-01-07        30
```

```
# Enregistrer le résultat dans l'objet "rx".
# Cette table sera utilisée pour des exemples plus tard
rx <- cohort_rx_dx %>% select(id, code, start, duration)
```

Exclusion

```
cohort_rx_dx %>% select(-dx1, -dx2, -dx3)
```

#>	id	sexe	rls	naiss	start	code	duration
#>	<int>	<char>	<char>	<Date>	<Date>	<char>	<num>
#> 1:	1	M	1111	1967-09-06	2000-01-03	C	1
#> 2:	1	M	1111	1967-09-06	2000-01-04	D	1
#> 3:	1	M	1111	1967-09-06	2000-01-07	S	30
#> 4:	1	M	1111	1967-09-06	2000-01-09	C	1
#> 5:	1	M	1111	1967-09-06	2000-01-11	C	1
#> 6:	1	M	1111	1967-09-06	2000-01-12	G	45

Commence par

```
cohort_rx_dx %>% select(id, code, start, starts_with("dx"))
```

```
#>      id  code      start  dx1  dx2  dx3
#>   <int> <char>   <Date> <char> <char> <char>
#> 1:     3  <NA> 2000-01-24    I    F  <NA>
#> 2:     3  <NA> 2000-02-18    B    G  <NA>
#> 3:     3  <NA> 2000-03-14    I  <NA>  <NA>
#> 4:     3  <NA> 2000-05-07    C    I    B
#> 5:     3    V 2000-05-25    D  <NA>  <NA>
#> 6:     3  <NA> 2000-08-15    E    B    F
```

Voir aussi `ends_with()`.

Contient

```
cohort_rx_dx %>% select(contains("is"))
```

```
#>      naiss  
#>      <Date>  
#> 1: 1967-09-06  
#> 2: 1967-09-06  
#> 3: 1967-09-06  
#> 4: 1967-09-06  
#> 5: 1967-09-06  
#> 6: 1967-09-06
```

Tout le reste

```
cohort_rx_dx %>% select(id, sexe, naiss, rls, everything())
```

```
#>      id  sexe    naiss    rls    start  code duration    dx1    dx2
#>   <int> <char>   <Date> <char>   <Date> <char>   <num> <char> <char>
#> 1:     1     M 1967-09-06  1111 2000-01-03     C      1    <NA>    <NA>
#> 2:     1     M 1967-09-06  1111 2000-01-04     D      1    <NA>    <NA>
#> 3:     1     M 1967-09-06  1111 2000-01-07     S     30    <NA>    <NA>
#> 4:     1     M 1967-09-06  1111 2000-01-09     C      1    <NA>    <NA>
#> 5:     1     M 1967-09-06  1111 2000-01-11     C      1    <NA>    <NA>
#> 6:     1     M 1967-09-06  1111 2000-01-12     G     45    <NA>    <NA>
#>      dx3
#>   <char>
#> 1:    <NA>
#> 2:    <NA>
#> 3:    <NA>
#> 4:    <NA>
#> 5:    <NA>
#> 6:    <NA>
```

Selon type

Numériques

```
cohort_rx_dx %>% select(where(is.numeric))
```

```
#>      id duration
#>   <int>   <num>
#> 1:      1       1
#> 2:      1       1
```

Caractères

```
cohort_rx_dx %>% select(where(is.character))
```

```
#>      sexe    rls    code    dx1    dx2    dx3
#>   <char> <char> <char> <char> <char> <char>
#> 1:      M   1111      C   <NA>   <NA>   <NA>
#> 2:      M   1111      D   <NA>   <NA>   <NA>
```

Voir aussi `is.factor`, `is.logical`, `is.Date`...

any_of / all_of

```
cols <- c("id", "sexe", "age", "rls", "code_postal")
```

any_of ignore les colonnes absentes

```
cohort_rx_dx %>% select(any_of(cols))
```

```
#>      id  sexe  rls  
#>   <int> <char> <char>  
#> 1:     1     M  1111  
#> 2:     1     M  1111
```

all_of s'assure que toutes les colonnes sont présentes en indiquant un message d'erreur si ce n'est pas le cas.

```
cohort_rx_dx %>% select(all_of(cols))
```

```
#> Error in `select()`:  
#> i In argument: `all_of(cols)`.  
#> Caused by error in `all_of()`:  
#> ! Can't subset elements that don't exist.  
#> ✖ Elements `age` and `code_postal` don't exist.
```

filter

On utilise `filter(.data, ...)` pour conserver les lignes et `filter_out(.data, ...)` pour les exclure.

Quelques exemples à utiliser dans `filter()` ou `filter_out()` :

```
<      # plus petit que  
>      # plus grand que  
==     # égal à  
%in%   # égal à plusieurs valeurs  
  
&      # et  
!      # contraire de  
  
is.na() # est valeur manquante
```

```
<=     # plus petit ou égal à  
>=     # plus grand ou égal à  
!=     # n'est pas égal à  
  
|      # ou  
  
between(x, y, z) # [y, z] de x
```

Condition simple

Toutes les prescriptions avec une durée (ou une quantité) de 30 jours.

```
cohort_rx_dx %>% select(id, sexe, naiss, code, duration) %>%  
  filter(duration == 30)
```

```
#>      id  sexe    naiss  code duration  
#>   <int> <char>   <Date> <char>    <num>  
#> 1:     1     M 1967-09-06     S      30  
#> 2:     1     M 1967-09-06     S      30  
#> 3:     1     M 1967-09-06     S      30  
#> 4:     1     M 1967-09-06     S      30  
#> 5:     1     M 1967-09-06     H      30  
#> ---  
#> 2564: 100     M 1990-11-03     B      30  
#> 2565: 100     M 1990-11-03     B      30  
#> 2566: 100     M 1990-11-03     B      30  
#> 2567: 100     M 1990-11-03     B      30  
#> 2568: 100     M 1990-11-03     B      30
```

Plusieurs conditions

- 1) Toutes les prescriptions avec une durée (ou une quantité) de 30 jours.
- 2) Exclure les RLS 1001, 1701 et 1801.
- 3) Au moins un diagnostic (dx1 a une valeur).

```
# Séparer par des virgules = ET / AND
cohort_rx_dx %>% select(id, sexe, rls, code, duration, dx1) %>%
  filter(duration == 30, !rls %in% c("1001", "1701", "1801"), !is.na(dx1))
```

```
#>      id  sexe  rls  code duration  dx1
#>   <int> <char> <char> <char>    <num> <char>
#> 1:     3     M  0811     W      30     J
#> 2:     6     M  0516     U      30     A
```

```
# Avec le symbole & = ET / AND
cohort_rx_dx %>% select(id, sexe, rls, code, duration, dx1) %>%
  filter(duration == 30 & !rls %in% c("1001", "1701", "1801") & !is.na(dx1))
```

```
#>      id  sexe  rls  code duration  dx1
#>   <int> <char> <char> <char>    <num> <char>
#> 1:     3     M  0811     W      30     J
#> 2:     6     M  0516     U      30     A
```

mutate

On utilise `mutate(.data, ...)` pour créer ou modifier des colonnes.

Créer une colonne

Indiquer la fin théorique de la prescription.

```
rx %>% mutate(end = start + duration - 1)
```

```
#>      id  code      start duration      end
#>   <int> <char>   <Date>     <num>   <Date>
#> 1:     1     C 2000-01-03         1 2000-01-03
#> 2:     1     D 2000-01-04         1 2000-01-04
#> 3:     1     S 2000-01-07        30 2000-02-05
```

Pour sauvegarder les résultats dans `rx` qui existe déjà :

```
rx <- rx %>% mutate(end = start + duration - 1)
```

Créer plusieurs colonnes

```
rx <- rx %>%
  mutate(
    end = start + duration - 1,
    qte_desc = case_when(
      duration == 1 ~ "unique",
      duration == 30 ~ "mensuel",
      duration == 90 ~ "trimestriel",
      is.na(duration) ~ NA,

      .default = "autre"
    ),
    # On peut utiliser une colonne créée précédemment
    mois_fin = lubridate::month(end)
  )
```

#>	id	code	start	duration	end	qte_desc	mois_fin
#>	<int>	<char>	<Date>	<num>	<Date>	<char>	<num>
#> 1:	1	C	2000-01-03	1	2000-01-03	unique	1
#> 2:	1	S	2000-01-07	30	2000-02-05	mensuel	2
#> 3:	1	C	2000-01-11	1	2000-01-11	unique	1
#> 4:	1	G	2000-01-12	45	2000-02-25	autre	2
#> 5:	1	Y	2000-01-19	90	2000-04-17	trimestriel	4

group_by

On utilise `group_by(.data, ...)` pour faire une analyse pour chaque groupe.

Supposons qu'on voudrait calculer, au moment du service, le nombre de jours qu'un individu s'est fait prescrire un médicament au moment (somme cumulative)

Sans `group_by`

```
rx[c(1:3, 298:300), c("id", "duration")] %>%  
  mutate(cumulatif = cumsum(duration))
```

```
#>      id duration cumulatif  
#>   <int>    <num>    <num>  
#> 1:     1        1         1  
#> 2:     1        1         2  
#> 3:     1       30        32  
#> 4:     2       45        77  
#> 5:     2       90       167  
#> 6:     2       60       227
```

Les 32 jours du id 1 s'ajoute aux premiers 45 jours du id 2.

Avec group_by

```
rx[c(1:3, 298:300), c("id", "duration")] %>%  
  group_by(id) %>%  
  mutate(cumulatif = cumsum(duration))
```

```
#>      id duration cumulatif  
#>   <int>    <num>    <num>  
#> 1:     1        1        1  
#> 2:     1        1        2  
#> 3:     1       30       32  
#> 4:     2       45       45  
#> 5:     2       90      135  
#> 6:     2       60      195
```

Dès qu'on arrive au id 2, la somme cumulative recommence.

ungroup

On utilise ungroup pour supprimer un group_by qui précède.

```
rx[c(1:3, 298:300)] %>%  
  select(id, start, duration) %>%  
  group_by(id) %>%  
  mutate(cumulatif = cumsum(duration),  
         min_start_id = min(start)) %>% # date min par id  
  ungroup() %>%  
  mutate(min_start_tout = min(start)) # date min pour tout
```

```
#> # A tibble: 6 × 6  
#>       id start      duration cumulatif min_start_id min_start_tout  
#>   <int> <date>      <dbl>      <dbl> <date>      <date>  
#> 1     1 2000-01-03         1         1 2000-01-03 2000-01-03  
#> 2     1 2000-01-04         1         2 2000-01-03 2000-01-03  
#> 3     1 2000-01-07        30        32 2000-01-03 2000-01-03  
#> 4     2 2000-01-19        45        45 2000-01-19 2000-01-03  
#> 5     2 2000-02-03        90       135 2000-01-19 2000-01-03  
#> 6     2 2000-02-07        60       195 2000-01-19 2000-01-03
```

summarise / summarize

On utilise `summarise(.data, ..., .by = NULL)` (ou `summarize()`) pour créer les statistiques.

```
rx %>%  
  filter(!is.na(duration)) %>%  
  mutate(annee = lubridate::year(start)) %>%  
  summarize(min = min(duration),  
            q1 = round(quantile(duration, 0.25), 1),  
            moy = round(mean(duration), 1),  
            med = median(duration),  
            q3 = round(quantile(duration, 0.75), 1),  
            max = max(duration),  
            n = n(),  
            .by = c("code", "annee"))
```

```
#>   code annee MIN Q1  MOY MEDIAN Q3 MAX  N  
#> 1    C  2000   1  7 30.4     30 60  90 341  
#> 2    D  2000   1  7 24.4     15 45  90 356  
#> 3    S  2000   1  7 41.1     30 90  90 302  
#> 4    G  2000   1 15 30.1     30 45  90 288  
#> 5    V  2000   1 15 42.1     45 60  90 414  
#> 6    Y  2000   1  7 36.2     30 60  90 302
```

arrange

On utilise `arrange(.data, ..., .by_group = FALSE)` pour trier les données dans l'ordre souhaité. Par défaut, les données sont triées en ordre croissant. On utilise `desc()` pour trier par ordre décroissant.

```
rx %>% arrange(id, code, desc(start))
```

```
#>           id   code      start duration
#>      <int> <char>   <Date>      <num>
#> 1:         1     B 2001-12-11         45
#> 2:         1     B 2001-11-29         45
#> 3:         1     B 2001-07-17         45
#> 4:         1     B 2001-07-15         45
#> 5:         1     B 2001-06-03         45
#> ---
#> 17192:    100      Z 2000-06-12          1
#> 17193:    100   <NA> 2001-06-29         NA
#> 17194:    100   <NA> 2000-09-06         NA
#> 17195:    100   <NA> 2000-05-05         NA
#> 17196:    100   <NA> 2000-03-14         NA
```

Après un summarize() :

```
rx %>%
  filter(!is.na(duration)) %>%
  mutate(annee = lubridate::year(start)) %>%
  summarize(MIN = min(duration),
            Q1 = round(quantile(duration, 0.25), 1),
            MOY = round(mean(duration), 1),
            MEDIAN = median(duration),
            Q3 = round(quantile(duration, 0.75), 1),
            MAX = max(duration),
            N = n(),

            .by = c("code", "annee")) %>%
  arrange(code, annee)
```

```
#>   code annee MIN   Q1  MOY MEDIAN Q3 MAX   N
#> 1    A  2000   1 30.0 49.1     45 90  90  236
#> 2    A  2001   1 22.5 49.3     45 90  90  215
#> 3    B  2000   1 15.0 37.3     30 60  90  394
#> 4    B  2001   1 15.0 38.7     45 60  90  427
#> 5    C  2000   1  7.0 30.4     30 60  90  341
#> 6    C  2001   1  7.0 33.3     30 60  90  328
```

left_join

On utilise le `left_join(x, y, by, ...)` pour ajouter des informations provenant d'une deuxième table.

Exemple

À partir de la table initiale, on construit une table de la cohorte d'étude :

```
cohort <- cohort_rx_dx %>%  
  select(id, sexe, naiss, rls) %>%  
  distinct()
```

```
#>      id  sexe    naiss    rls  
#>   <int> <char>   <Date> <char>  
#> 1:     1     M 1967-09-06   1111  
#> 2:     2     F 1962-09-10   0652  
#> 3:     3     M 1970-09-17   0811  
#> 4:     4     F 1962-06-05   1214  
#> 5:     5     M 1966-07-31   0814  
#> 6:     6     M 1979-12-15   0516
```

À partir du fichier *Estimations et projections de population par territoire sociosanitaire* du Ministère de la Santé et des Services sociaux, on s'est construit une table de dimension RLS.

```
head(dim_rls)
```

```
#>      CODE      DESCRIPTIF
#>      <char>      <char>
#> 1:    0111      RLS de Kamouraska
#> 2:    0112 RLS de Rivière-du-Loup
#> 3:    0113      RLS de Témiscouata
#> 4:    0114      RLS des Basques
#> 5:    0115      RLS de Rimouski
#> 6:    0116      RLS de La Mitis
```

Grâce à `left_join` on est en mesure d'ajouter les informations de la table `dim_rls` à notre table `cohort`.

```
cohort %>%  
  left_join(dim_rls, by = c("rls" = "CODE")) %>%  
  rename(nom_rls = DESCRIPTIF)
```

```
#>      id  sexe    naiss    rls  
#>   <int> <char>   <Date> <char>  
#> 1:     1     M 1967-09-06   1111  
#> 2:     2     F 1962-09-10   0652  
#> 3:     3     M 1970-09-17   0811  
#> 4:     4     F 1962-06-05   1214  
#> 5:     5     M 1966-07-31   0814  
#> 6:     6     M 1979-12-15   0516  
#>                                     nom_rls  
#>                                     <char>  
#> 1:                                     RLS de la Baie-des-Chaleurs  
#> 2: RLS de Rivière-des-Prairies - Anjou - Montréal-Est  
#> 3:                                     RLS de l'Abitibi-Ouest  
#> 4:                                     RLS des Etchemins  
#> 5:                                     RLS de la Vallée-de-l'Or  
#> 6:                                     RLS de Val Saint-François
```

Voir aussi :

- `inner_join()`
- `right_join()`
- `full_join()`

Cheatsheet

Une *cheatsheet* est une fiche de référence qui regroupe les fonctions et les commandes les plus utiles.

Il est possible de les consulter sur le site du *tidyverse*.

Consulter une *cheatsheet* n'est pas une preuve qu'on ne connaît pas R. C'est la preuve qu'on n'essaie pas de mémoriser des centaines de fonctions pour rien.

À retenir : **ALLER LES CONSULTER!!!**

tidyr

Quand utiliser tidyr ?

On utilise `tidyr` lorsqu'on veut modifier **la structure** d'une table, **le contenant**, alors que `dplyr` était pour le contenu.

La page officielle est disponible [ici](#).

`tidyr` contient combien d'objets (fonctions, variables, tables...) ?

```
library(tidyr)
length(ls("package:tidyr"))
```

```
#> [1] 89
```

pivot_longer

Il est préférable (pour ne pas dire obligé) de travailler avec des tables *en longueur* où chaque ligne est **une seule et unique observation**.

Exemple où on a plusieurs diagnostics sur la même ligne.

```
# Table des diagnostics avec les informations pertinentes
dx <- cohort_rx_dx %>%
  select(id, start, dx1, dx2, dx3) %>%
  filter(!is.na(dx1)) # lignes sans diagnostics
```

```
#>      id      start  dx1  dx2  dx3
#>   <int>   <Date> <char> <char> <char>
#> 1:      3 2000-01-24     I     F  <NA>
#> 2:      3 2000-02-18     B     G  <NA>
#> 3:      3 2000-03-14     I  <NA>  <NA>
#> 4:      3 2000-05-07     C     I     B
#> 5:      3 2000-05-25     D  <NA>  <NA>
#> 6:      3 2000-08-15     E     B     F
```

Les colonnes dx1, dx2 et dx3 ne respectent pas le concept de *une ligne par observation*. On transforme la table pour que chaque colonne devienne une ligne.

```
dx_long <- dx %>%  
  pivot_longer(  
    cols = dx1:dx3,  
    names_to = "ndx",  
    values_to = "dx",  
    values_drop_na = TRUE  
  )
```

```
#>   id      start ndx dx  
#> 1  3 2000-01-24 dx1  I  
#> 2  3 2000-01-24 dx2  F  
#> 3  3 2000-02-18 dx1  B  
#> 4  3 2000-02-18 dx2  G  
#> 5  3 2000-03-14 dx1  I  
#> 6  3 2000-05-07 dx1  C
```

R a été conçu pour manipuler des vecteurs. Comme les colonnes d'une table sont des vecteurs, R est généralement plus efficace lorsqu'on effectue une opération sur une colonne complète plutôt que lorsqu'on traite les données ligne par ligne.

pivot_wider

Contraire de `pivot_longer()` où on préfère passer d'une table longue à une table large.

Exemple où on voudrait créer un tableau de présence à partir de la table suivante :

```
presence_id
```

```
#>   id data_source
#> 1  1          RAMQ
#> 2  2          RAMQ
#> 3  2    MED_ECHO
#> 4  3          RAMQ
#> 5  3    MED_ECHO
#> 6  3          DECES
```

On veut indiquer la présence de l'individu par 1 et son absence par 0.

```
presence_id %>%  
  mutate(present = 1)
```

```
#>   id data_source present  
#> 1  1          RAMQ      1  
#> 2  2          RAMQ      1  
#> 3  2    MED_ECHO      1  
#> 4  3          RAMQ      1  
#> 5  3    MED_ECHO      1  
#> 6  3          DECES      1
```

```
presence_id %>%  
  mutate(present = 1) %>%  
  pivot_wider(  
    names_from = data_source,  
    values_from = present,  
    values_fill = 0  
  )
```

```
#> # A tibble: 3 × 4  
#>       id  RAMQ MED_ECHO DECES  
#>   <dbl> <dbl>   <dbl> <dbl>  
#> 1     1     1       0     0  
#> 2     2     1       1     0  
#> 3     3     1       1     1
```

tidyr > pivot_wider

stringr

Quand utiliser stringr ?

On utilise `stringr` pour la manipulation des données. Sa principale mission est de faciliter la manipulation des chaînes de caractères :

```
x <- "  Espaces au début ? À la fin ?  "  
gsub("^\\s+|\\s+$", "", x) # R de base
```

```
#> [1] "Espaces au début ? À la fin ?"
```

```
stringr::str_trim(x) # stringr
```

```
#> [1] "Espaces au début ? À la fin ?"
```

La page officielle est disponible [ici](#).

`stringr` contient combien d'objets (fonctions, variables, tables...) ?

```
library(stringr)  
length(ls("package:stringr"))
```

```
#> [1] 66
```

Fonctions utiles

Détection

```
str_detect(c("Diabète", "Asthme"), "Diab")
```

```
#> [1] TRUE FALSE
```

Remplacement

```
str_replace("A-B-C", "-", "_")
```

```
#> [1] "A_B-C"
```

```
str_replace_all("A-B-C", "-", "_")
```

```
#> [1] "A_B_C"
```

Longueur

```
str_length("Québec")
```

```
#> [1] 6
```

Concaténation

```
str_c("Patient_", 123) # équivalent de paste0()
```

```
#> [1] "Patient_123"
```

Sous-chaîne

```
str_sub("2025-09-01", 1, 4)
```

```
#> [1] "2025"
```

Majuscules et minuscules

```
str_to_upper("Diabète")
```

```
#> [1] "DIABÈTE"
```

```
str_to_lower("Diabète")
```

```
#> [1] "diabète"
```

Retrait d'espaces

```
str_trim(" Québec ")
```

```
#> [1] "Québec"
```

```
str_squish(" Vivre au Québec ") # str_trim + str_replace_all
```

```
#> [1] "Vivre au Québec"
```

stringr dans dplyr

En reprenant notre exemple de *diagnostics long* :

```
#>   id      start ndx dx
#> 1  3 2000-01-24 dx1  I
#> 2  3 2000-01-24 dx2  F
#> 3  3 2000-02-18 dx1  B
#> 4  3 2000-02-18 dx2  G
```

Je veux que la colonne `ndx` soit un nombre entier allant de 1 à N au lieu de `dx1` à `dxN`.

```
dx_long <- dx_long %>%
  mutate(ndx = as.integer(str_remove(ndx, "dx")))
```

```
#>   id      start ndx dx
#> 1  3 2000-01-24   1  I
#> 2  3 2000-01-24   2  F
#> 3  3 2000-02-18   1  B
#> 4  3 2000-02-18   2  G
```

lubridate

Quand utiliser lubridate ?

On peut utiliser `lubridate` aussitôt qu'on travaille avec une variable de type `Date`.

Le package `lubridate` simplifie le code en voulant exprimer le temps comme un concept plutôt qu'une chaîne de caractère formatée.

```
as.integer(format(as.Date("2020-06-01"), "%Y")) # R de base
```

```
#> [1] 2020
```

```
year(as_date("2020-06-01")) # lubridate
```

```
#> [1] 2020
```

La page officielle est disponible [ici](#).

`lubridate` contient combien d'objets (fonctions, variables, tables...) ?

```
library(lubridate)
length(ls("package:lubridate"))
```

```
#> [1] 163
```

Fonctions utiles

Créer une date

```
ymd("2025-06-15")
```

```
#> [1] "2025-06-15"
```

```
dmy("15-06-2025")
```

```
#> [1] "2025-06-15"
```

Extraire des composantes

```
year(ymd("2025-06-15"))
```

```
#> [1] 2025
```

```
month(ymd("2025-06-15"))
```

```
#> [1] 6
```

Ajouter ou soustraire du temps

```
ymd("2025-06-15") + days(30)
```

```
#> [1] "2025-07-15"
```

```
# Attention aux 29 février !!!  
ymd("2024-02-29") - months(24)
```

```
#> [1] NA
```

```
ymd("2024-02-29") %m-% months(24) # %m+$ pour ajouter
```

```
#> [1] "2022-02-28"
```

Période actuelle ou suivante

```
floor_date(ymd("2025-06-15"), "month")
```

```
#> [1] "2025-06-01"
```

```
ceiling_date(ymd("2025-06-15"), "month")
```

```
#> [1] "2025-07-01"
```

readr

Quand utiliser readr ?

Le *package* readr est la boîte à outils spécialisée pour lire les fichiers texte (csv, txt, tsv) dans R.

La page officielle est disponible [ici](#).

readr contient combien d'objets (fonctions, variables, tables...) ?

```
library(readr)
length(ls("package:readr"))
```

```
#> [1] 103
```

Si on utilise les fonctions de base en R qui commencent par read. pour importer ou write. pour exporter, leur fonction équivalente en readr sont read_ et write_.

Par exemple read.csv() devient read_csv().

Fonctions utiles

CSV

```
read_csv(); write_csv()  # séparateur "," et décimale "."  
read_csv2(); write_csv2() # séparateur ";" et décimale ",",
```

TSV

```
read_tsv(); write_tsv()  # tsv = tab-separated values (\t)
```

Fichier général

```
read_delim(); write_delim() # fonction générale où on décide  
                             # le délimiteur et autres paramètres
```

Autres *packages*

- `haven` pour du SPSS, Stata et SAS.
- `readxl` et `writexl` pour des fichiers Excel.
Il y a d'autres solutions, mais essayer d'éviter ceux qui sont dépendants de Java.
- `data.table` avec `fread` et `fwrite`.
- `fst` est un format dédié à R où on combine performance, compression et sélection des colonnes et des lignes.

Pour finir

- N'oubliez pas d'aller voir la documentation des fonctions!
Ils contiennent souvent des exemples de code qu'on peut exécuter.
- **Les Vignettes**
Les vignettes sont des guides pratiques fournis avec un *package*.
Contrairement à l'aide d'une fonction qui explique une commande précise, une vignette montre généralement comment utiliser plusieurs fonctions ensemble pour réaliser une tâche complète.